

Instructions for Submission

1. File Naming and Submission:
 - a. Create a **ZIP file** containing all your solution files.
 - b. Name the ZIP file as lab06_<ERP>.zip, where <ERP> is your ERP.
 - c. Ensure that the ZIP file contains:
 - i. C++ source files
 - ii. No unnecessary files (such as executables, temporary files, or editor configs)
 - d. Do not include TEXT(.txt) files.
2. Code Neatness:
 - a. Ensure your code is **well-formatted** and easy to read. Use proper **indentation** and **meaningful variable names**.
 - b. Include appropriate **comments** to explain the logic where necessary (especially for functions and complex sections of the code).
 - c. **Add clear explanations where required (or specified)** to make the logic of your program easy to follow.
3. Code Compilation and Functionality:
 - a. Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
 - b. Double-check that the functionality of the program meets the requirements outlined in the task.
4. Additional Instructions:
 - a. Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
 - b. Make sure your code adheres to the principles of good software development (e.g., minimal repetition, and no hard-coding).
 - c. If you encounter any issues or require clarifications, reach out before the submission deadline.
5. Deadline:
 - a. Ensure that you submit the assignment on time. Late submissions will not be accepted.
 - b. Double-check that the zip file contains all required files before submitting.
6. Plagiarism Policy:
 - a. The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Review

Two-Dimensional Arrays

In programming, we often need to represent information that naturally falls into rows and columns. A chessboard, a digital image, a multiplication table, or even a seating arrangement can all be modeled as a grid. C++ provides a way to represent such data structures using two-dimensional arrays.

A two-dimensional array is essentially an array of arrays. The general form of declaration is:

```
data_type array_name [rows][columns];
```

For example:

```
int a[3][4] = {  
    {2, 5, 7, 1},  
    {1, 3, 6, 1},  
    {5, 4, 1, 3}  
};
```

This creates an array with 3 rows and 4 columns. The element at the second row and third column is accessed as `a[1][2]`.

Internally, the elements are stored in *row-major order*, which means that all elements of the first row are stored in memory first, then the second row, and so on. This has performance implications in scenarios such as matrix multiplication, where accessing elements row by row is more efficient than by columns.

Two-dimensional arrays are the foundation for many advanced problems in computer science, including image processing, graph algorithms (via adjacency matrices), and scientific computing. Even though larger and more flexible data structures exist, understanding how to manipulate static 2D arrays is essential, since it teaches memory layout, indexing, and iteration using nested loops.

Sorting: Insertion Sort and Selection Sort

Sorting is the process of arranging elements into a defined order, most commonly ascending or descending. Sorting is not just an academic exercise—it is fundamental to many computer systems. File systems rely on sorting for indexing, search engines use sorting to rank results, and databases constantly sort records to optimize queries.

Two classical algorithms that illustrate how sorting can be performed are **insertion sort** and **selection sort**.

- **Insertion Sort:** This algorithm builds the sorted list one element at a time. Each new element is inserted into its proper place among the previously sorted elements, which may require shifting larger elements to the right. The worst-case scenario occurs when the input array is in descending order, because every new element has to be shifted across all the already-sorted elements.
- **Selection Sort:** This algorithm repeatedly selects the smallest element from the unsorted part of the array and places it at the beginning. Unlike insertion sort, the number of swaps is independent of how the input is arranged. The worst-case scenario for selection sort is the same as the best case: no matter the initial order, it will still perform one swap per pass through the array.

Although both algorithms are inefficient for very large datasets compared to modern algorithms like quicksort or mergesort, they are easy to implement and analyze. They also illustrate different trade-offs: insertion sort adapts to input order and is faster when data is nearly sorted, while selection sort makes fewer swaps but cannot take advantage of partially ordered data.

Vectors (std::vector)

Static arrays in C++ require the size to be fixed at compile time. In practice, however, we often do not know in advance how many items will need to be stored. For example, when parsing input, logging events, or reading unknown amounts of data, the container must grow as new items arrive.

The C++ Standard Library provides `std::vector` as a dynamic array type. A vector automatically resizes itself as elements are added or removed, while still allowing fast random access like a normal array. To use vectors, the header `<vector>` must be included.

General Syntax and Operations:

- `std::vector<data type> vec name;` → declares a vector of the given data type

- `vec_name.push_back(value);` → adds an element at the end of the vector
- `vec_name.size();` → returns the number of elements currently in the vector
- `vec_name[i];` → accesses the element at index `i`

Example:

```
std::vector<int> v;  
v.push_back(5);  
v.push_back(8);  
std::cout << "Size: " << v.size(); // prints 2  
std::cout << "First element: " << v[0]; // prints 5
```

Vectors combine the best of arrays (direct indexing) with the flexibility of dynamic growth. They form the basis of more advanced containers and are heavily used in competitive programming, system utilities, and real-world applications where datasets vary in size.

Strings (`std::string`)

Strings are sequences of characters used to represent text. Unlike primitive arrays of characters, the `std::string` class provides built-in functionality to manipulate text safely and conveniently. Strings are central to programming, as virtually every application must process text—whether it is user input, source code, messages, or identifiers.

General Syntax:

```
std::string s;
```

Common Operations:

- `s.size()` → returns number of characters in the string
- `s[i]` → accesses character at position `i`
- `s.substr(i, n)` → returns substring starting at `i` with length `n`
- `s.find(t)` → returns index of first occurrence of substring `t`, or `std::string::npos` if not found
- `s.append(t)` → appends string `t` to the end of `s`
- `s + t` → concatenates two strings

Example:

```
std::string name = "abcabc";  
std::cout << name.substr(2,3); // prints "cab"  
if (name.find("abc") != std::string::npos)  
    std::cout << "Pattern found";
```

Internally, a string is stored as an array of characters, which means you can still access and process it like a character array, but with much greater convenience. This combination of low-level accessibility and high-level functionality makes strings one of the most important classes in C++.

Lab Tasks

Task 1a: 2D Matrix

You need to declare an array of 5 x 9 elements. In which first 1st column contains the StudentID and its respective 5 courses obtained marks in following 5 five columns for the semester. In the rest of 4 columns total marks, obtained marks, and Percentage as shown in given table.

- Initialize the array with its default value.
- You need to take user input for the first 6 columns.
- Total marks, obtained marks, and Percentage columns will be filled by your program based on required logic for each of the columns.

Id	C1	C2	C3	C4	C5	Total	Obtained	Percentage

Task 1b: Matrix Multiplication

You have two small matrices, A (Sales Figures) and B (Profit Multipliers). To find the total Weighted Sales Contribution, you must calculate the product matrix $C=A \times B$.

Matrix A (Sales: 2×3): Represents sales data for 2 regions across 3 product categories.

$$A = \begin{pmatrix} 10 & 20 & 5 \\ 15 & 10 & 8 \end{pmatrix}$$

Matrix B (Profit Multipliers: 3×2): Represents the profit multiplier for the 3 product categories across 2 different market conditions.

$$B = \begin{pmatrix} 2 & 1 \\ 3 & 4 \\ 1 & 2 \end{pmatrix}$$

Workflow:

1. **Declare and Initialize** a 2×3 integer array for Matrix A and a 3×2 integer array for Matrix B.
2. **Declare** a 2×2 integer array named **ResultMatrix** to store the product $C=A \times B$. (The resulting matrix size must be 2×2).
3. **Implement the matrix multiplication algorithm** using **three nested loops**.

- The core calculation is:

$$C[i][j] = \sum_{k=0}^2 (A[i][k] \times B[k][j])$$

4. **Print** the final 2×2 **ResultMatrix** neatly.

Task 2a: Insertion Sort

Declare a 1D array of random integers. Sort the array and count the number times an integer is swapped and repeat this for array sizes of 1 through 10. Output the arrays and the List of swaps with each array size.

Task 2b: Selection Sort

Declare a 1D array of random integers. Sort the array and count the number times an integer is swapped and repeat this for array sizes of 1 through 10. Output the arrays and the List of swaps with each array size.

Compare both Sorts and tell which is most efficient. Give your reasons why.

Task 3: Vectors

You are collecting an unknown number of test scores. Since **std::vector** handles dynamic size, you'll use it to gather scores until a specific exit condition is met.

Workflow:

- **Declare** an `std::vector<int>` named Scores.
- Use a **do-while loop** to repeatedly prompt the user to enter a positive integer score.
- Add the score to the vector using `push_back()`.
- The loop must terminate when the user enters the number **-1** (do not add -1 to the vector).
- After the loop finishes, print:
 - The **size** of the final vector.
 - The **first element** in the vector (index 0)
 - The **last element** added to the vector (using the size to calculate the index).